

DATA ANALYSIS & AI FUNDAMENTALS

Trainer: Lumumba W. Victor

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load and explore data
df = pd.read_csv('data.csv')
print(df.head())
print(df.describe())

# Data preprocessing
df = df.dropna()
X = df.drop('target', axis=1)
y = df['target']

# Train test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Evaluate
score = model.score(X_test, y_test)
print(f"Model Accuracy: {score:.2f}")
```

DATA IMPORTATION, CLEANING AND WRANGLING/MANIPULATION

1. DATA IMPORTATION

1.1. Install and Load the Required Libraries

In [1]:

```
#!pip install pandas
#!pip install numpy
#!pip install matplotlib
#!pip install seaborn
```

1.2. Loading the Libraries

In [2]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sn

## For data importation and handling data f
## For handling numeric operations
## For data visualization
## For data visualization

### Filter warnings
import warnings
warnings.filterwarnings("ignore")
```

1.3. Reading the Dataset into Jupyter Notebook

In [3]:

```
df = pd.read_csv("GSSsubset1.csv")
```

1.4. General Data Inspection

1.4.1. Check the Column Names

In [4]:

```
print(f"Columns: {df.columns.tolist()}")
```

Columns: ['id', 'sex', 'degree', 'income', 'marital', 'age', 'height', 'weight', 'hrswork']

1.4.2. Check the Shape of the Data

In [5]:

```
print(f"Shape: {df.shape}")
```

Shape: (3000, 9)

1.4.3. Check the Variable Types

In [6]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   id           3000 non-null   int64  
1   sex          3000 non-null   str     
2   degree       2942 non-null   str     
3   income       2767 non-null   float64 
4   marital      2934 non-null   str     
5   age          3000 non-null   int64  
6   height       3000 non-null   float64 
7   weight       2879 non-null   float64 
8   hrswork      2933 non-null   float64 
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

1.4.4. Check the First Few Observations

In [7]:

```
df.head(20)
```

Out[7]:

	id	sex	degree	income	marital	age	height	weight	hrswork
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0
5	6	FEMALE	HIGH SCHOOL	47790.38	MARRIED	43	60.7	68.4	46.0
6	7	FEMALE	BACHELOR	102903.23	MARRIED	42	66.5	66.7	51.0

	id	sex	degree	income	marital	age	height	weight	hrswrk
7	8	MALE	HIGH SCHOOL	36707.09	NEVER MARRIED	52	65.9	76.8	48.0
8	9	FEMALE	BACHELOR	48404.51	MARRIED	35	66.8	94.6	28.0
9	10	MALE	HIGH SCHOOL	29044.70	NEVER MARRIED	18	69.3	85.2	NaN
10	11	FEMALE	GRADUATE	419380.86	SEPARATED	58	62.4	64.8	31.0
11	12	MALE	JUNIOR COLLEGE	14995.63	WIDOWED	40	66.8	68.7	45.0
12	13	FEMALE	LT HIGH SCHOOL	22832.15	WIDOWED	52	61.3	61.0	25.0
13	14	MALE	LT HIGH SCHOOL	23183.92	NaN	50	71.6	100.4	55.0
14	15	FEMALE	GRADUATE	114886.28	MARRIED	40	64.1	75.5	42.0
15	16	FEMALE	BACHELOR	59232.59	MARRIED	51	62.9	72.9	20.0
16	17	MALE	BACHELOR	NaN	NEVER MARRIED	24	69.3	84.9	50.0
17	18	FEMALE	HIGH SCHOOL	37509.86	SEPARATED	49	61.5	58.8	31.0
18	19	FEMALE	LT HIGH SCHOOL	20512.23	WIDOWED	60	59.0	62.9	22.0
19	20	FEMALE	BACHELOR	84696.68	DIVORCED	63	59.7	65.5	42.0

1.4.5.View the last Few Obervations

In [8]:

```
df.tail(10)
```

Out[8]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
2990	2991	FEMALE	BACHELOR	47934.48	MARRIED	42	64.6	62.9	36.0
2991	2992	MALE	HIGH SCHOOL	NaN	MARRIED	53	70.0	83.7	46.0
2992	2993	MALE	GRADUATE	308066.05	MARRIED	37	68.6	92.4	61.0
2993	2994	MALE	HIGH SCHOOL	NaN	MARRIED	58	66.5	86.3	35.0
2994	2995	FEMALE	HIGH SCHOOL	69153.23	DIVORCED	49	62.1	59.7	39.0
2995	2996	MALE	HIGH SCHOOL	42319.59	MARRIED	60	69.3	85.8	51.0
2996	2997	MALE	GRADUATE	175090.66	MARRIED	57	69.7	98.5	51.0
2997	2998	FEMALE	LT HIGH SCHOOL	12813.84	WIDOWED	51	62.2	62.8	42.0
2998	2999	FEMALE	GRADUATE	61419.36	NEVER MARRIED	18	61.2	60.4	48.0
2999	3000	FEMALE	HIGH SCHOOL	78015.07	MARRIED	33	66.3	72.8	8.0

1.5. General Descriptive Statistics

In [9]:

```
df.describe()
```

Out[9]:

	id	income	age	height	weight	hrswrk
count	3000.000000	2767.000000	3000.000000	3000.000000	2879.000000	2933.000000
mean	1500.500000	63871.815790	42.078000	67.400500	76.389232	39.778043
std	866.169729	51634.075898	13.314316	3.641297	14.584233	12.594723
min	1.000000	3720.040000	18.000000	58.100000	45.000000	0.000000
25%	750.750000	31989.025000	33.000000	64.600000	65.800000	34.000000
50%	1500.500000	49586.540000	42.000000	67.200000	75.700000	41.000000
75%	2250.250000	78986.595000	51.000000	70.100000	85.700000	48.000000
max	3000.000000	609741.180000	80.000000	78.600000	120.000000	79.000000

In [10]:

```
### Remove the id Column from the Summary Statistics
df.drop(columns = ["id"]).describe()
```

Out[10]:

	income	age	height	weight	hrswrk
count	2767.000000	3000.000000	3000.000000	2879.000000	2933.000000
mean	63871.815790	42.078000	67.400500	76.389232	39.778043
std	51634.075898	13.314316	3.641297	14.584233	12.594723
min	3720.040000	18.000000	58.100000	45.000000	0.000000
25%	31989.025000	33.000000	64.600000	65.800000	34.000000
50%	49586.540000	42.000000	67.200000	75.700000	41.000000
75%	78986.595000	51.000000	70.100000	85.700000	48.000000
max	609741.180000	80.000000	78.600000	120.000000	79.000000

1.6. General Frequency Tables

In [11]:

```
print("="*110)
print("Distribution of Gender in the General Social Survey Data")
print("="*110)

df["sex"].value_counts()
```

```
=====
Distribution of Gender in the General Social Survey Data
=====
```

Out[11]:

```
sex
FEMALE    1563
MALE      1437
Name: count, dtype: int64
```

In [12]:

```
print("="*110)
print("Distribution of Education Qualification in the General Social Survey Data")
print("="*110)

df["degree"].value_counts()
```

```
=====
=====
Distribution of Education Qualification in the General Social Survey Data
=====
=====
```

```
Out[12]:
degree
HIGH SCHOOL      1094
LT HIGH SCHOOL   583
BACHELOR         566
JUNIOR COLLEGE   465
GRADUATE         234
Name: count, dtype: int64
```

```
In [13]:
print("="*110)
print("Distribution of Marital Status in the General Social Survey Data")
print("="*110)

df["marital"].value_counts()
```

```
=====
=====
Distribution of Marital Status in the General Social Survey Data
=====
=====
```

```
Out[13]:
marital
MARRIED          1518
NEVER MARRIED     691
DIVORCED          382
WIDOWED           227
SEPARATED         116
Name: count, dtype: int64
```

2. DATA CLEANING

2.1. Handling Missing Values

```
In [14]:
df.head(20)
```

```
Out[14]:
```

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0

	id	sex	degree	income	marital	age	height	weight	hrswrk
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0
5	6	FEMALE	HIGH SCHOOL	47790.38	MARRIED	43	60.7	68.4	46.0
6	7	FEMALE	BACHELOR	102903.23	MARRIED	42	66.5	66.7	51.0
7	8	MALE	HIGH SCHOOL	36707.09	NEVER MARRIED	52	65.9	76.8	48.0
8	9	FEMALE	BACHELOR	48404.51	MARRIED	35	66.8	94.6	28.0
9	10	MALE	HIGH SCHOOL	29044.70	NEVER MARRIED	18	69.3	85.2	NaN
10	11	FEMALE	GRADUATE	419380.86	SEPARATED	58	62.4	64.8	31.0
11	12	MALE	JUNIOR COLLEGE	14995.63	WIDOWED	40	66.8	68.7	45.0
12	13	FEMALE	LT HIGH SCHOOL	22832.15	WIDOWED	52	61.3	61.0	25.0
13	14	MALE	LT HIGH SCHOOL	23183.92	NaN	50	71.6	100.4	55.0
14	15	FEMALE	GRADUATE	114886.28	MARRIED	40	64.1	75.5	42.0
15	16	FEMALE	BACHELOR	59232.59	MARRIED	51	62.9	72.9	20.0
16	17	MALE	BACHELOR	NaN	NEVER MARRIED	24	69.3	84.9	50.0
17	18	FEMALE	HIGH SCHOOL	37509.86	SEPARATED	49	61.5	58.8	31.0
18	19	FEMALE	LT HIGH SCHOOL	20512.23	WIDOWED	60	59.0	62.9	22.0
19	20	FEMALE	BACHELOR	84696.68	DIVORCED	63	59.7	65.5	42.0

2.1.1. Check the Presence of Missing Values

In [15]:

```
print("="*110)
print("Checking the Presence of Missing Values in the General Social Survey Data")
print("="*110)

print(df.isna().sum())
```

```
=====
Checking the Presence of Missing Values in the General Social Survey Data
=====
```

```
=====
id          0
sex         0
degree     58
income     233
marital     66
age         0
height      0
weight     121
hrswrk      67
dtype: int64
```

In [16]:

```
print("="*110)
print("Checking the Percentage of Missing Values per Variable in the General Social Survey Data")
print("="*110)
```

```
Missing_values = df.isna().mean().round(4)*100
Missing_values
```

```
=====
Checking the Percentage of Missing Values per Variable in the General Social Survey Data
=====
```

```
Out[16]:
id          0.00
sex         0.00
degree      1.93
income      7.77
marital     2.20
age         0.00
height      0.00
weight      4.03
hrswrk      2.23
dtype: float64
```

2.1.2. Methods of Handling Missing Values

- Complete cases Analysis- Commonly used approach but Risky
- Mean and/or Median Imputation - For symmetric and asymmetric data
- Mode Imputation - For Categorical and Character data

2.1.3. Complete Cases Analysis (Row Deletion)

```
In [17]:
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   id          3000 non-null   int64  
 1   sex         3000 non-null   str     
 2   degree      2942 non-null   str     
 3   income      2767 non-null   float64 
 4   marital     2934 non-null   str     
 5   age         3000 non-null   int64  
 6   height      3000 non-null   float64 
 7   weight      2879 non-null   float64 
 8   hrswrk      2933 non-null   float64 
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

```
In [18]:
```

```
df_drop_all = df.dropna()
print(df_drop_all.isna().sum())
```

```
id          0
sex          0
degree       0
income       0
marital      0
age          0
height       0
```

```
weight      0
hrswrk      0
dtype: int64
```

In [19]:

```
df_drop_all.info()
```

```
<class 'pandas.DataFrame'>
Index: 2503 entries, 0 to 2999
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id           2503 non-null   int64
1   sex          2503 non-null   str
2   degree       2503 non-null   str
3   income       2503 non-null   float64
4   marital      2503 non-null   str
5   age          2503 non-null   int64
6   height       2503 non-null   float64
7   weight       2503 non-null   float64
8   hrswrk       2503 non-null   float64
dtypes: float64(4), int64(2), str(3)
memory usage: 195.5 KB
```

2.1.4. Mean - Imputation (for symmetric data)

In [20]:

```
## Identify the Numeric Columns
df_mean = df.copy()
df_mean.head()
```

Out[20]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0

In [21]:

```
df_mean.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id           3000 non-null   int64
1   sex          3000 non-null   str
2   degree       2942 non-null   str
3   income       2767 non-null   float64
4   marital      2934 non-null   str
5   age          3000 non-null   int64
6   height       3000 non-null   float64
7   weight       2879 non-null   float64
8   hrswrk       2933 non-null   float64
```

dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB

In [22]:

```
numeric_cols = df_mean.select_dtypes(include = "number").columns  
df_mean[numeric_cols] = df_mean[numeric_cols].fillna(df_mean[numeric_cols].mean())
```

In [23]:

```
print(df.isna().sum())
```

```
id          0  
sex         0  
degree     58  
income     233  
marital     66  
age         0  
height      0  
weight     121  
hrswrk      67  
dtype: int64
```

In [24]:

```
print(df_mean.isna().sum())
```

```
id          0  
sex         0  
degree     58  
income      0  
marital     66  
age         0  
height      0  
weight      0  
hrswrk      0  
dtype: int64
```

2.1.5. Median Imputation for Asymmetric Data

In [25]:

```
df_median = df.copy()  
df_median.head()
```

Out[25]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0

In [26]:

```
numeric_cols = df_median.select_dtypes(include = "number").columns  
df_median[numeric_cols] = df_median[numeric_cols].fillna(df_median[numeric_cols].median())
```

In [27]:

```
print(df_median.isna().sum())
```

```
id          0
sex          0
degree      58
income       0
marital      66
age          0
height       0
weight       0
hrswrk       0
dtype: int64
```

2.1.5. Mode Imputation for Categorical and Character Data

In [28]:

```
df["degree"].value_counts()
```

Out[28]:

```
degree
HIGH SCHOOL      1094
LT HIGH SCHOOL    583
BACHELOR         566
JUNIOR COLLEGE   465
GRADUATE         234
Name: count, dtype: int64
```

In [29]:

```
df["sex"].value_counts()
```

Out[29]:

```
sex
FEMALE    1563
MALE      1437
Name: count, dtype: int64
```

In [30]:

```
df["marital"].value_counts()
```

Out[30]:

```
marital
MARRIED      1518
NEVER MARRIED 691
DIVORCED      382
WIDOWED       227
SEPARATED     116
Name: count, dtype: int64
```

In [31]:

```
cat_cols = df.select_dtypes(include = ["object", "category"]).columns
for col in cat_cols:
    mode_val = df[col].mode()
    if not mode_val.empty:
        df[col] = df[col].fillna(mode_val[0])
```

In [32]:

```
df.isna().sum()
```

Out[32]:

```
id          0
sex          0
degree       0
income      233
```

```
marital      0
age          0
height       0
weight      121
hrswrk       67
dtype: int64
```

In [33]:

```
df["degree"].value_counts()
```

Out[33]:

```
degree
HIGH SCHOOL      1152
LT HIGH SCHOOL    583
BACHELOR         566
JUNIOR COLLEGE   465
GRADUATE         234
Name: count, dtype: int64
```

In [34]:

```
numeric_cols = df.select_dtypes(include = "number").columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
```

2.1.7. Check the Presence of Missing Values after Imputation

In [35]:

```
df.isna().sum()
```

Out[35]:

```
id          0
sex         0
degree      0
income      0
marital     0
age         0
height      0
weight      0
hrswrk      0
dtype: int64
```

2.1.8. Check the Data Structure

In [36]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id          3000 non-null   int64
1   sex         3000 non-null   str
2   degree      3000 non-null   str
3   income      3000 non-null   float64
4   marital     3000 non-null   str
5   age         3000 non-null   int64
6   height      3000 non-null   float64
7   weight      3000 non-null   float64
8   hrswrk      3000 non-null   float64
```

dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB

2.2. Handling the Duplicate Entries

2.2.1. Check the Duplicate Entries

In [37]:

```
dir(df)
```

Out[37]:

```
['T',  
 '_AXIS_LEN',  
 '_AXIS_ORDERS',  
 '_AXIS_TO_AXIS_NUMBER',  
 '_HANDLED_TYPES',  
 '__abs__',  
 '__add__',  
 '__and__',  
 '__annotations__',  
 '__array__',  
 '__array_priority__',  
 '__array_ufunc__',  
 '__arrow_c_stream__',  
 '__bool__',  
 '__class__',  
 '__contains__',  
 '__copy__',  
 '__dataframe__',  
 '__deepcopy__',  
 '__delattr__',  
 '__delitem__',  
 '__dict__',  
 '__dir__',  
 '__divmod__',  
 '__doc__',  
 '__eq__',  
 '__finalize__',  
 '__floordiv__',  
 '__format__',  
 '__ge__',  
 '__getattr__',  
 '__getattribute__',  
 '__getitem__',  
 '__getstate__',  
 '__gt__',  
 '__hash__',  
 '__iadd__',  
 '__iand__',  
 '__ifloordiv__',  
 '__imod__',  
 '__imul__',  
 '__init__',  
 '__init_subclass__',  
 '__invert__',  
 '__ior__',  
 '__ipow__',  
 '__isub__',  
 '__iter__',
```

```
'__itruediv__',
'__ixor__',
'__le__',
'__len__',
'__lt__',
'__matmul__',
'__mod__',
'__module__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
'_accessors',
'_accum_func',
'_agg_examples_doc',
'_agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append_internal',
'_arith_method',
'_arith_method_with_reindex',
'_attrs',
'_box_col_values',
'_can_fast_transpose',
'_check_copy_deprecation',
'_check_inplace_and_allows_duplicate_labels',
'_check_label_or_level_ambiguity',
'_clip_with_one_bound',
```

'_clip_with_scalar',
'_cmp_method',
'_combine_frame',
'_consolidate',
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
'_constructor',
'_constructor_from_mgr',
'_constructor_sliced',
'_constructor_sliced_from_mgr',
'_dir_additions',
'_dir_deletions',
'_dispatch_frame_op',
'_drop_axis',
'_drop_labels_or_levels',
'_ensure_valid_index',
'_find_valid_index',
'_flags',
'_flex_arith_method',
'_flex_cmp_method',
'_from_arrays',
'_from_mgr',
'_get_agg_axis',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
'_get_cleaned_column_resolvers',
'_get_column_array',
'_get_index_resolvers',
'_get_item',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_value',
'_get_values_for_csv',
'_getitem_bool_array',
'_getitem_multilevel',
'_getitem_slice',
'_gotitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_info_repr',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_homogeneous_type',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_iset_item',
'_iset_item_mgr',

```
'_iset_not_inplace',
'_iter_column_arrays',
'_ixs',
'_logical_func',
'_logical_method',
'_maybe_align_series_as_frame',
'_metadata',
'_mgr',
'_min_count_stat_function',
'_module_source',
'_needs_reindex_multi',
'_pad_or_backfill',
'_reduce',
'_reduce_axis1',
'_reindex_axes',
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
'_replace_columnwise',
'_repr_data_resource',
'_repr_fits_horizontal',
'_repr_fits_vertical',
'_repr_html',
'_repr_latex',
'_reset_cache',
'_sanitize_column',
'_select_dtypes_indices',
'_series',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_item',
'_set_item_frame_value',
'_set_item_mgr',
'_set_value',
'_setitem_array',
'_setitem_frame',
'_setitem_slice',
'_shift_with_freq',
'_should_reindex_frame_op',
'_slice',
'_stat_function',
'_stat_function_ddof',
'_to_dict_of_blocks',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'age',
'agg',
'aggregate',
'align',
'all',
```

'any',
'apply',
'asfreq',
'asof',
'assign',
'astype',
'at',
'at_time',
'attrs',
'axes',
'between_time',
'bfill',
'boxplot',
'clip',
'columns',
'combine',
'combine_first',
'compare',
'convert_dtypes',
'copy',
'corr',
'corrwith',
'count',
'cov',
'cummax',
'cummin',
'cumprod',
'cumsum',
'degree',
'describe',
'diff',
'div',
'divide',
'dot',
'drop',
'drop_duplicates',
'droplevel',
'dropna',
'dtypes',
'duplicated',
'empty',
'eq',
'equals',
'eval',
'ewm',
'expanding',
'explode',
'ffill',
'fillna',
'filter',
'first_valid_index',
'flags',
'floordiv',
'from_arrow',
'from_dict',
'from_records',
'ge',
'get',
'groupby',

'gt',
'head',
'height',
'hist',
'hrswrk',
'iat',
'id',
'idxmax',
'idxmin',
'iloc',
'income',
'index',
'infer_objects',
'info',
'insert',
'interpolate',
'isetitem',
'isin',
'isna',
'isnull',
'items',
'iterrows',
'itertuples',
'join',
'keys',
'kurt',
'kurtosis',
'last_valid_index',
'le',
'loc',
'lt',
'map',
'marital',
'mask',
'max',
'mean',
'median',
'melt',
'memory_usage',
'merge',
'min',
'mod',
'mode',
'mul',
'multiply',
'ndim',
'ne',
'nlargest',
'notna',
'notnull',
'nsmallest',
'nunique',
'pct_change',
'pipe',
'pivot',
'pivot_table',
'plot',
'pop',
'pow',

'prod',
'product',
'quantile',
'query',
'radd',
'rank',
'rdiv',
'reindex',
'reindex_like',
'rename',
'rename_axis',
'reorder_levels',
'replace',
'resample',
'reset_index',
'rfloordiv',
'rmod',
'rmul',
'rolling',
'round',
'rpow',
'rsub',
'rtruediv',
'sample',
'select_dtypes',
'sem',
'set_axis',
'set_flags',
'set_index',
'sex',
'shape',
'shift',
'size',
'skew',
'sort_index',
'sort_values',
'squeeze',
'stack',
'std',
'style',
'sub',
'subtract',
'sum',
'swaplevel',
'tail',
'take',
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_feather',
'to_hdf',
'to_html',
'to_iceberg',
'to_json',
'to_latex',
'to_markdown',
'to_numpy',
'to_orc',

```
'to_parquet',
'to_period',
'to_pickle',
'to_records',
'to_sql',
'to_stata',
'to_string',
'to_timestamp',
'to_xarray',
'to_xml',
'transform',
'transpose',
'truediv',
'truncate',
'tz_convert',
'tz_localize',
'unstack',
'update',
'value_counts',
'values',
'var',
'weight',
'where',
'xs']
```

In [38]:

```
dup_rows = df.duplicated().sum()
print(dup_rows)
```

0

2.2.1. Drop the Duplicate Entries if any Exists

In [39]:

```
clean_df = df.drop_duplicates()
clean_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   id          3000 non-null   int64  
 1   sex         3000 non-null   str     
 2   degree      3000 non-null   str     
 3   income      3000 non-null   float64 
 4   marital     3000 non-null   str     
 5   age         3000 non-null   int64  
 6   height      3000 non-null   float64 
 7   weight      3000 non-null   float64 
 8   hrswrk      3000 non-null   float64 
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

2.3. Handling Outliers

Outliers are values that lie abnormally very low or very high (far off the trend)

In [40]:

```
df.head(40)
```

```
Out[40]:
```

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56000	NEVER MARRIED	18	73.2	86.300000	35.000000
1	2	MALE	HIGH SCHOOL	36544.91000	MARRIED	35	69.9	95.300000	47.000000
2	3	FEMALE	HIGH SCHOOL	59587.14000	DIVORCED	28	70.1	60.900000	16.000000
3	4	FEMALE	GRADUATE	63871.81579	MARRIED	43	61.9	76.000000	44.000000
4	5	MALE	HIGH SCHOOL	48899.47000	MARRIED	48	65.8	58.900000	44.000000
5	6	FEMALE	HIGH SCHOOL	47790.38000	MARRIED	43	60.7	68.400000	46.000000
6	7	FEMALE	BACHELOR	102903.23000	MARRIED	42	66.5	66.700000	51.000000
7	8	MALE	HIGH SCHOOL	36707.09000	NEVER MARRIED	52	65.9	76.800000	48.000000
8	9	FEMALE	BACHELOR	48404.51000	MARRIED	35	66.8	94.600000	28.000000
9	10	MALE	HIGH SCHOOL	29044.70000	NEVER MARRIED	18	69.3	85.200000	39.778043
10	11	FEMALE	GRADUATE	419380.86000	SEPARATED	58	62.4	64.800000	31.000000
11	12	MALE	JUNIOR COLLEGE	14995.63000	WIDOWED	40	66.8	68.700000	45.000000
12	13	FEMALE	LT HIGH SCHOOL	22832.15000	WIDOWED	52	61.3	61.000000	25.000000
13	14	MALE	LT HIGH SCHOOL	23183.92000	MARRIED	50	71.6	100.400000	55.000000
14	15	FEMALE	GRADUATE	114886.28000	MARRIED	40	64.1	75.500000	42.000000
15	16	FEMALE	BACHELOR	59232.59000	MARRIED	51	62.9	72.900000	20.000000
16	17	MALE	BACHELOR	63871.81579	NEVER MARRIED	24	69.3	84.900000	50.000000
17	18	FEMALE	HIGH SCHOOL	37509.86000	SEPARATED	49	61.5	58.800000	31.000000
18	19	FEMALE	LT HIGH SCHOOL	20512.23000	WIDOWED	60	59.0	62.900000	22.000000
19	20	FEMALE	BACHELOR	84696.68000	DIVORCED	63	59.7	65.500000	42.000000
20	21	FEMALE	JUNIOR COLLEGE	42356.40000	MARRIED	35	63.6	61.800000	31.000000
21	22	FEMALE	HIGH SCHOOL	58293.37000	NEVER MARRIED	38	67.1	82.400000	30.000000
22	23	FEMALE	HIGH SCHOOL	21576.95000	DIVORCED	57	66.7	97.700000	39.778043
23	24	FEMALE	HIGH SCHOOL	31959.51000	DIVORCED	34	62.8	58.100000	40.000000
24	25	FEMALE	JUNIOR COLLEGE	59242.09000	MARRIED	50	62.5	63.100000	42.000000
25	26	FEMALE	JUNIOR COLLEGE	51360.70000	NEVER MARRIED	39	63.5	52.400000	46.000000

	id	sex	degree	income	marital	age	height	weight	hrswrk
26	27	FEMALE	BACHELOR	110137.83000	MARRIED	32	65.6	69.600000	53.000000
27	28	FEMALE	HIGH SCHOOL	25667.25000	MARRIED	43	65.8	77.500000	34.000000
28	29	FEMALE	LT HIGH SCHOOL	121734.28000	MARRIED	40	65.3	76.389232	52.000000
29	30	MALE	JUNIOR COLLEGE	58158.35000	WIDOWED	36	74.1	90.200000	23.000000
30	31	MALE	JUNIOR COLLEGE	84068.32000	NEVER MARRIED	34	63.3	63.200000	47.000000
31	32	FEMALE	HIGH SCHOOL	63871.81579	MARRIED	52	66.4	81.600000	27.000000
32	33	MALE	LT HIGH SCHOOL	43002.77000	WIDOWED	67	72.0	103.300000	40.000000
33	34	FEMALE	LT HIGH SCHOOL	28402.42000	MARRIED	36	68.0	85.400000	43.000000
34	35	MALE	LT HIGH SCHOOL	41715.79000	DIVORCED	25	69.0	85.000000	29.000000
35	36	FEMALE	HIGH SCHOOL	140892.25000	MARRIED	29	63.7	52.900000	36.000000
36	37	FEMALE	LT HIGH SCHOOL	24677.38000	NEVER MARRIED	45	61.6	74.500000	33.000000
37	38	FEMALE	GRADUATE	166733.37000	NEVER MARRIED	42	65.8	68.800000	59.000000
38	39	MALE	LT HIGH SCHOOL	50400.92000	MARRIED	27	68.8	80.300000	1.000000
39	40	FEMALE	JUNIOR COLLEGE	63871.81579	MARRIED	45	66.0	63.200000	52.000000

2.3.1. Check the Presence of Outliers

In [41]:

```
numeric_cols = df.select_dtypes(include = ["float64", "int64"]).columns
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"Columns '{col}': {len(outliers)} Outliers")
```

```
Columns 'id': 0 Outliers
Columns 'income': 201 Outliers
Columns 'age': 10 Outliers
Columns 'height': 2 Outliers
Columns 'weight': 32 Outliers
Columns 'hrswrk': 155 Outliers
```

2.3.2. Print the IDs Where the Values are Considered Outliers

In [42]:

```
numeric_cols = ["income", "age", "height", "weight", "hrswrk"]
outlier_id = {}
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    outlier_id[col] = outliers["id"].tolist()

print(f"Column '{col}': {len(outliers)} outliers")
print(outliers['id'].tolist())
print("-"*110)
```

Column 'income': 201 outliers

[11, 36, 38, 73, 76, 87, 88, 102, 111, 112, 144, 146, 174, 200, 206, 211, 223, 227, 244, 275, 286, 289, 306, 326, 352, 359, 375, 381, 397, 404, 437, 455, 460, 493, 505, 519, 523, 536, 544, 549, 568, 591, 596, 603, 608, 660, 668, 727, 730, 731, 732, 761, 767, 786, 797, 808, 834, 840, 847, 913, 923, 931, 946, 975, 976, 981, 984, 1003, 1034, 1054, 1061, 1082, 1087, 1093, 1108, 1127, 1128, 1134, 1149, 1163, 1189, 1242, 1250, 1310, 1344, 1407, 1414, 1418, 1461, 1489, 1506, 1573, 1575, 1598, 1609, 1627, 1638, 1659, 1676, 1677, 1681, 1685, 1691, 1694, 1696, 1710, 1711, 1717, 1731, 1760, 1762, 1770, 1805, 1806, 1814, 1819, 1867, 1870, 1873, 1886, 1888, 1916, 1943, 1953, 1961, 1965, 2018, 2019, 2030, 2042, 2075, 2088, 2107, 2120, 2148, 2149, 2153, 2170, 2181, 2228, 2246, 2254, 2265, 2268, 2281, 2285, 2287, 2292, 2295, 2325, 2336, 2374, 2397, 2414, 2462, 2473, 2475, 2480, 2486, 2498, 2527, 2531, 2539, 2550, 2579, 2590, 2607, 2617, 2627, 2633, 2634, 2644, 2645, 2696, 2701, 2708, 2716, 2728, 2732, 2760, 2769, 2785, 2789, 2801, 2802, 2809, 2820, 2828, 2830, 2844, 2858, 2864, 2880, 2896, 2897, 2933, 2948, 2954, 2979, 2993, 2997]

Column 'age': 10 outliers

[1254, 1550, 1555, 1717, 1959, 2094, 2242, 2327, 2734, 2838]

Column 'height': 2 outliers

[257, 431]

Column 'weight': 32 outliers

[202, 222, 279, 326, 382, 451, 522, 523, 683, 708, 820, 958, 998, 1083, 1374, 1430, 1556, 1600, 1640, 1681, 1742, 1958, 2015, 2219, 2234, 2235, 2396, 2462, 2575, 2842, 2870, 2898]

Column 'hrswrk': 155 outliers

[39, 56, 63, 67, 72, 73, 106, 107, 115, 124, 208, 246, 250, 260, 278, 282, 305, 322, 378, 385, 387, 424, 426, 433, 437, 469, 479, 506, 535, 581, 585, 600, 602, 622, 658, 672, 694, 734, 736, 777, 780, 823, 848, 865, 936, 983, 996, 1029, 1057, 1087, 1129, 1150, 1194, 1196, 1197, 1223, 1229, 1248, 1274, 1287, 1302, 1305, 1316, 1335, 1345, 1350, 1386, 1388, 1406, 1426, 1429, 1439, 1450, 1466, 1568, 1571, 1585, 1612, 1624, 1627, 1682, 1731, 1739, 1742, 1756, 1762, 1769, 1790, 1792, 1799, 1803, 1819, 1822, 1832, 1879, 1884, 1891, 1894, 1898, 1902, 1944, 1992, 2019, 2071, 2130, 2178, 2237, 2239, 2265, 2272, 2300, 2317, 2333, 2337, 2343, 2352, 2370, 2413, 2419, 2448, 2470, 2505, 2522, 2524, 2532, 2534,

2538, 2549, 2550, 2555, 2568, 2569, 2607, 2629, 2640, 2649, 2664, 2684, 2685, 2689, 2707, 2721, 2732, 2748, 2796, 2851, 2854, 2901, 2917, 2946, 2951, 2958, 2968, 2981, 3000]

2.3.3. Cap the Outliers

In [43]:

```
numeric_cols = df.select_dtypes(include = ["float64", "int64"]).columns
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    ## Cap the Outliers
    df[col] = np.where(df[col] < lower_bound, lower_bound, df[col])
    df[col] = np.where(df[col] > upper_bound, upper_bound, df[col])
```

2.3.4. Check the Presence of Outliers After Capping

In [44]:

```
numeric_cols = df.select_dtypes(include = ["float64", "int64"]).columns
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"Columns '{col}': {len(outliers)} Outliers")
```

Columns 'id': 0 Outliers
Columns 'income': 0 Outliers
Columns 'age': 0 Outliers
Columns 'height': 0 Outliers
Columns 'weight': 0 Outliers
Columns 'hrswrk': 0 Outliers

2.4. View the Data After Cleaning

In [45]:

```
df.head(10)
```

Out[45]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1.0	MALE	HIGH SCHOOL	26722.56000	NEVER MARRIED	18.0	73.2	86.3	35.000000
1	2.0	MALE	HIGH SCHOOL	36544.91000	MARRIED	35.0	69.9	95.3	47.000000
2	3.0	FEMALE	HIGH SCHOOL	59587.14000	DIVORCED	28.0	70.1	60.9	16.000000
3	4.0	FEMALE	GRADUATE	63871.81579	MARRIED	43.0	61.9	76.0	44.000000
4	5.0	MALE	HIGH SCHOOL	48899.47000	MARRIED	48.0	65.8	58.9	44.000000

	id	sex	degree	income	marital	age	height	weight	hswrk
5	6.0	FEMALE	HIGH SCHOOL	47790.38000	MARRIED	43.0	60.7	68.4	46.000000
6	7.0	FEMALE	BACHELOR	102903.23000	MARRIED	42.0	66.5	66.7	51.000000
7	8.0	MALE	HIGH SCHOOL	36707.09000	NEVER MARRIED	52.0	65.9	76.8	48.000000
8	9.0	FEMALE	BACHELOR	48404.51000	MARRIED	35.0	66.8	94.6	28.000000
9	10.0	MALE	HIGH SCHOOL	29044.70000	NEVER MARRIED	18.0	69.3	85.2	39.778043

2.5. Save the Data as Clean Dataset

In [46]:

```
df.to_csv("Cleaned_gss_data.csv")
```

3. DATA MANIPULATION

- Selecting (Column Selection)
- Filtering (Subsetting Rows)
- groupby
- Aggregating and Summarizing
- Combining Filter and Selection

In [47]:

```
df = pd.read_csv("GSSsubset.csv")
df.head()
```

Out[47]:

	id	sex	degree	income	marital	age	height	weight	hswrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6

3.1. FILTERING

Subset rows that meet certain conditions

In [48]:

```
### Create male participants data
male = df[df['sex'] == 'MALE']
male.head()
```

Out[48]:

	id	sex	degree	income	marital	age	height	weight	hswrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60

	id	sex	degree	income	marital	age	height	weight	hrrwrk
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6
5	19	MALE	LT HIGH SCHOOL	15703.75	MARRIED	51	68	170	50
7	27	MALE	HIGH SCHOOL	17551.25	SEPARATED	35	70	180	40
8	28	MALE	BACHELOR	73900.00	MARRIED	57	71	225	40

3.2. MULTIPLE FILTERING

In [49]:

```
subset = df[(df['sex'] == 'FEMALE') &
            (df['age'] < 35) &
            (df['marital'] == "MARRIED") &
            (df['degree'] == "GRADUATE")]
subset.head()
```

Out[49]:

	id	sex	degree	income	marital	age	height	weight	hrrwrk
18	49	FEMALE	GRADUATE	88680.0	MARRIED	32	66	160	40
115	312	FEMALE	GRADUATE	60967.5	MARRIED	30	66	150	47
412	1037	FEMALE	GRADUATE	49882.5	MARRIED	30	61	129	50
747	1920	FEMALE	GRADUATE	88680.0	MARRIED	31	67	180	40
856	2209	FEMALE	GRADUATE	88680.0	MARRIED	34	66	148	30

3.3. OR CONDITION

In [50]:

```
df.head(20)
```

Out[50]:

	id	sex	degree	income	marital	age	height	weight	hrrwrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6
5	19	MALE	LT HIGH SCHOOL	15703.75	MARRIED	51	68	170	50
6	21	FEMALE	HIGH SCHOOL	17551.25	DIVORCED	30	62	115	38
7	27	MALE	HIGH SCHOOL	17551.25	SEPARATED	35	70	180	40
8	28	MALE	BACHELOR	73900.00	MARRIED	57	71	225	40
9	30	MALE	BACHELOR	33255.00	DIVORCED	54	71	165	40
10	32	FEMALE	BACHELOR	33255.00	DIVORCED	61	64	128	40

	id	sex	degree	income	marital	age	height	weight	hrswrk
11	38	MALE	HIGH SCHOOL	33255.00	DIVORCED	31	67	150	39
12	40	MALE	BACHELOR	73900.00	MARRIED	35	69	200	50
13	44	MALE	HIGH SCHOOL	24017.50	NEVER MARRIED	26	76	165	45
14	45	FEMALE	BACHELOR	88680.00	SEPARATED	50	67	188	60
15	46	FEMALE	HIGH SCHOOL	33255.00	NEVER MARRIED	43	68	150	40
16	47	MALE	JUNIOR COLLEGE	40645.00	NEVER MARRIED	30	72	185	45
17	48	MALE	HIGH SCHOOL	49882.50	MARRIED	59	70	175	40
18	49	FEMALE	GRADUATE	88680.00	MARRIED	32	66	160	40
19	50	MALE	HIGH SCHOOL	24017.50	MARRIED	32	66	170	40

In [51]:

```
high_educ = df[(df['degree'] == "BACHELOR") |
               (df['degree'] == "GRADUATE")]

high_educ.head(20)
```

Out[51]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	BACHELOR	60967.500	DIVORCED	53	72	190	60
1	2	FEMALE	BACHELOR	60967.500	MARRIED	26	60	97	40
2	4	FEMALE	BACHELOR	10161.250	MARRIED	56	68	160	20
8	28	MALE	BACHELOR	73900.000	MARRIED	57	71	225	40
9	30	MALE	BACHELOR	33255.000	DIVORCED	54	71	165	40
10	32	FEMALE	BACHELOR	33255.000	DIVORCED	61	64	128	40
12	40	MALE	BACHELOR	73900.000	MARRIED	35	69	200	50
14	45	FEMALE	BACHELOR	88680.000	SEPARATED	50	67	188	60
18	49	FEMALE	GRADUATE	88680.000	MARRIED	32	66	160	40
20	51	FEMALE	GRADUATE	33255.000	MARRIED	52	63	143	40
21	52	MALE	BACHELOR	40645.000	MARRIED	42	68	230	40
23	56	FEMALE	BACHELOR	1478.000	NEVER MARRIED	24	65	118	16
24	59	FEMALE	BACHELOR	73900.000	NEVER MARRIED	56	66	148	30
25	61	MALE	GRADUATE	15703.750	NEVER MARRIED	51	70	190	14
26	64	FEMALE	GRADUATE	60967.500	NEVER MARRIED	47	64	123	40
27	68	MALE	GRADUATE	158656.952	DIVORCED	57	71	180	70
29	75	FEMALE	GRADUATE	88680.000	MARRIED	39	69	210	60
30	77	MALE	BACHELOR	103460.000	MARRIED	54	70	200	60
37	102	MALE	GRADUATE	49882.500	NEVER MARRIED	32	75	180	55
38	106	MALE	BACHELOR	33255.000	MARRIED	41	65	170	50

3.4. SELECTION

Columns Selection

3.4.1. Single Column Selection

In [52]:

```
income_col = df['income']  
income_col.head()
```

Out[52]:

```
0    60967.50  
1    60967.50  
2    10161.25  
3    17551.25  
4    17551.25  
Name: income, dtype: float64
```

3.4.2. Select Multiple Columns

In [53]:

```
### First Option  
cols = ["sex", "age", "income", "marital"]  
selected_cols = df[cols]  
selected_cols.head()
```

Out[53]:

	sex	age	income	marital
0	MALE	53	60967.50	DIVORCED
1	FEMALE	26	60967.50	MARRIED
2	FEMALE	56	10161.25	MARRIED
3	FEMALE	40	17551.25	MARRIED
4	MALE	56	17551.25	MARRIED

In [54]:

```
#### Second Option  
selected_cols = df[["sex", "age", "income", "marital"]]  
selected_cols.head()
```

Out[54]:

	sex	age	income	marital
0	MALE	53	60967.50	DIVORCED
1	FEMALE	26	60967.50	MARRIED
2	FEMALE	56	10161.25	MARRIED
3	FEMALE	40	17551.25	MARRIED
4	MALE	56	17551.25	MARRIED

In [55]:

```
dir(df)
```

Out[55]:

```
['T',
 '_AXIS_LEN',
 '_AXIS_ORDERS',
 '_AXIS_TO_AXIS_NUMBER',
 '_HANDLED_TYPES',
 '__abs__',
 '__add__',
 '__and__',
 '__annotations__',
 '__array__',
 '__array_priority__',
 '__array_ufunc__',
 '__arrow_c_stream__',
 '__bool__',
 '__class__',
 '__contains__',
 '__copy__',
 '__dataframe__',
 '__deepcopy__',
 '__delattr__',
 '__delitem__',
 '__dict__',
 '__dir__',
 '__divmod__',
 '__doc__',
 '__eq__',
 '__finalize__',
 '__floordiv__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__iand__',
 '__ifloordiv__',
 '__imod__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__invert__',
 '__ior__',
 '__ipow__',
 '__isub__',
 '__iter__',
 '__itruediv__',
 '__ixor__',
 '__le__',
 '__len__',
 '__lt__',
 '__matmul__',
 '__mod__',
```

```
'__module__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
'_accessors',
'_accum_func',
'_agg_examples_doc',
'_agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append_internal',
'_arith_method',
'_arith_method_with_reindex',
'_attrs',
'_box_col_values',
'_can_fast_transpose',
'_check_copy_deprecation',
'_check_inplace_and_allows_duplicate_labels',
'_check_label_or_level_ambiguity',
'_clip_with_one_bound',
'_clip_with_scalar',
'_cmp_method',
'_combine_frame',
'_consolidate',
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
```

```
'_constructor',
'_constructor_from_mgr',
'_constructor_sliced',
'_constructor_sliced_from_mgr',
'_dir_additions',
'_dir_deletions',
'_dispatch_frame_op',
'_drop_axis',
'_drop_labels_or_levels',
'_ensure_valid_index',
'_find_valid_index',
'_flags',
'_flex_arith_method',
'_flex_cmp_method',
'_from_arrays',
'_from_mgr',
'_get_agg_axis',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
'_get_cleaned_column_resolvers',
'_get_column_array',
'_get_index_resolvers',
'_get_item',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_value',
'_get_values_for_csv',
'_getitem_bool_array',
'_getitem_multilevel',
'_getitem_slice',
'_gotitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_info_repr',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_homogeneous_type',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_iset_item',
'_iset_item_mgr',
'_iset_not_inplace',
'_iter_column_arrays',
'_ixs',
'_logical_func',
'_logical_method',
'_maybe_align_series_as_frame',
'_metadata',
```

'_mgr',
'_min_count_stat_function',
'_module_source',
'_needs_reindex_multi',
'_pad_or_backfill',
'_reduce',
'_reduce_axis1',
'_reindex_axes',
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
'_replace_columnwise',
'_repr_data_resource',
'_repr_fits_horizontal',
'_repr_fits_vertical',
'_repr_html',
'_repr_latex',
'_reset_cache',
'_sanitize_column',
'_select_dtypes_indices',
'_series',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_item',
'_set_item_frame_value',
'_set_item_mgr',
'_set_value',
'_setitem_array',
'_setitem_frame',
'_setitem_slice',
'_shift_with_freq',
'_should_reindex_frame_op',
'_slice',
'_stat_function',
'_stat_function_ddof',
'_to_dict_of_blocks',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'age',
'agg',
'aggregate',
'align',
'all',
'any',
'apply',
'asfreq',
'asof',
'assign',
'astype',
'at',

'at_time',
'attrs',
'axes',
'between_time',
'bfill',
'boxplot',
'clip',
'columns',
'combine',
'combine_first',
'compare',
'convert_dtypes',
'copy',
'corr',
'corrwith',
'count',
'cov',
'cummax',
'cummin',
'cumprod',
'cumsum',
'degree',
'describe',
'diff',
'div',
'divide',
'dot',
'drop',
'drop_duplicates',
'dropna',
'dropna',
'dtypes',
'duplicated',
'empty',
'eq',
'equals',
'eval',
'ewm',
'expanding',
'explode',
'ffill',
'fillna',
'filter',
'first_valid_index',
'flags',
'floordiv',
'from_arrow',
'from_dict',
'from_records',
'ge',
'get',
'groupby',
'gt',
'head',
'height',
'hist',
'hrswrk',
'iat',
'id',

'idxmax',
'idxmin',
'iloc',
'income',
'index',
'infer_objects',
'info',
'insert',
'interpolate',
'isetitem',
'isin',
'isna',
'isnull',
'items',
'iterrows',
'itertuples',
'join',
'keys',
'kurt',
'kurtosis',
'last_valid_index',
'le',
'loc',
'lt',
'map',
'marital',
'mask',
'max',
'mean',
'median',
'melt',
'memory_usage',
'merge',
'min',
'mod',
'mode',
'mul',
'multiply',
'ndim',
'ne',
'nlargest',
'notna',
'notnull',
'nsmallest',
'nunique',
'pct_change',
'pipe',
'pivot',
'pivot_table',
'plot',
'pop',
'pow',
'prod',
'product',
'quantile',
'query',
'radd',
'rank',
'rdiv',

'reindex',
'reindex_like',
'rename',
'rename_axis',
'reorder_levels',
'replace',
'resample',
'reset_index',
'rfloordiv',
'rmod',
'rmul',
'rolling',
'round',
'rpow',
'rsub',
'rtruediv',
'sample',
'select_dtypes',
'sem',
'set_axis',
'set_flags',
'set_index',
'sex',
'shape',
'shift',
'size',
'skew',
'sort_index',
'sort_values',
'squeeze',
'stack',
'std',
'style',
'sub',
'subtract',
'sum',
'swaplevel',
'tail',
'take',
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_feather',
'to_hdf',
'to_html',
'to_iceberg',
'to_json',
'to_latex',
'to_markdown',
'to_numpy',
'to_orc',
'to_parquet',
'to_period',
'to_pickle',
'to_records',
'to_sql',
'to_stata',
'to_string',

```
'to_timestamp',
'to_xarray',
'to_xml',
'transform',
'transpose',
'truediv',
'truncate',
'tz_convert',
'tz_localize',
'unstack',
'update',
'value_counts',
'values',
'var',
'weight',
'where',
'xs']
```

In [56]:

```
#### Third Option
drop_cols = df.drop(columns = ["id", "degree", "height", "hrswrk", "weight"])
drop_cols.head()
```

Out[56]:

	sex	income	marital	age
0	MALE	60967.50	DIVORCED	53
1	FEMALE	60967.50	MARRIED	26
2	FEMALE	10161.25	MARRIED	56
3	FEMALE	17551.25	MARRIED	40
4	MALE	17551.25	MARRIED	56

3.5. GROUP BY AND SUMMARIZE

In [57]:

```
print("="*110)
print("GROUP BY AND SUMMARIZING")
print("="*110)

print("Mean Income by Education Levels")

income_by_educ = df.groupby('degree')['income'].mean().sort_values(ascending = False)
income_by_educ
```

```
=====
=====
GROUP BY AND SUMMARIZING
=====
=====
```

Mean Income by Education Levels

Out[57]:

degree	
GRADUATE	68457.773552
BACHELOR	47738.621187
JUNIOR COLLEGE	30580.141304

HIGH SCHOOL 28309.892098
LT HIGH SCHOOL 18412.616883
Name: income, dtype: float64

In [58]:

```
dir(income_by_educ)
```

Out[58]:

```
['BACHELOR',  
'GRADUATE',  
'T',  
'_AXIS_LEN',  
'_AXIS_ORDERS',  
'_AXIS_TO_AXIS_NUMBER',  
'_HANDLED_TYPES',  
'__abs__',  
'__add__',  
'__and__',  
'__annotations__',  
'__array__',  
'__array_priority__',  
'__array_ufunc__',  
'__arrow_c_stream__',  
'__bool__',  
'__class__',  
'__contains__',  
'__copy__',  
'__deepcopy__',  
'__delattr__',  
'__delitem__',  
'__dict__',  
'__dir__',  
'__divmod__',  
'__doc__',  
'__eq__',  
'__finalize__',  
'__floordiv__',  
'__format__',  
'__ge__',  
'__getattr__',  
'__getattribute__',  
'__getitem__',  
'__getstate__',  
'__gt__',  
'__hash__',  
'__iadd__',  
'__iand__',  
'__ifloordiv__',  
'__imod__',  
'__imul__',  
'__init__',  
'__init_subclass__',  
'__invert__',  
'__ior__',  
'__ipow__',  
'__isub__',  
'__iter__',  
'__itruediv__',  
'__ixor__']
```

```
'__le__',
'__len__',
'__lt__',
'__matmul__',
'__mod__',
'__module__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
'_accessors',
'_accum_func',
'_agg_examples_doc',
'_agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append_internal',
'_arith_method',
'_attrs',
'_binop',
'_can_hold_na',
'_check_copy_deprecation',
'_check_inplace_and_allows_duplicate_labels',
'_check_label_or_level_ambiguity',
'_clip_with_one_bound',
'_clip_with_scalar',
'_cmp_method',
'_consolidate',
```

```
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
'_constructor',
'_constructor_expanddim',
'_constructor_expanddim_from_mgr',
'_constructor_from_mgr',
'_dir_additions',
'_dir_deletions',
'_drop_axis',
'_drop_labels_or_levels',
'_duplicated',
'_find_valid_index',
'_flags',
'_flex_method',
'_from_mgr',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
'_get_cleaned_column_resolvers',
'_get_index_resolvers',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_rows_with_mask',
'_get_value',
'_get_values_tuple',
'_get_with',
'_getitem_slice',
'_gotitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_init_dict',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_ixs',
'_logical_func',
'_logical_method',
'_map_values',
'_memory_usage',
'_metadata',
'_mgr',
'_min_count_stat_function',
'_module_source',
'_name',
'_needs_reindex_multi',
'_pad_or_backfill',
'_reduce',
```

'_references',
'_reindex_axes',
'_reindex_indexer',
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
'_repr_data_resource_',
'_repr_latex_',
'_reset_cache',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_labels',
'_set_name',
'_set_value',
'_set_values',
'_set_with',
'_set_with_engine',
'_shift_with_freq',
'_slice',
'_stat_function',
'_stat_function_ddof',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'agg',
'aggregate',
'align',
'all',
'any',
'apply',
'argmax',
'argmin',
'argsort',
'array',
'asfreq',
'asof',
'astype',
'at',
'at_time',
'attrs',
'autocorr',
'axes',
'between',
'between_time',
'bfill',
'case_when',
'clip',
'combine',
'combine_first',
'compare',
'convert_dtypes',

'copy',
'corr',
'count',
'cov',
'cummax',
'cummin',
'cumprod',
'cumsum',
'describe',
'diff',
'div',
'divide',
'divmod',
'dot',
'drop',
'drop_duplicates',
'droplevel',
'dropna',
'dtype',
'dtypes',
'duplicated',
'empty',
'eq',
'equals',
'ewm',
'expanding',
'explode',
'factorize',
'ffill',
'fillna',
'filter',
'first_valid_index',
'flags',
'floordiv',
'from_arrow',
'ge',
'get',
'groupby',
'gt',
'hasnans',
'head',
'hist',
'iat',
'idxmax',
'idxmin',
'iloc',
'index',
'infer_objects',
'info',
'interpolate',
'is_monotonic_decreasing',
'is_monotonic_increasing',
'is_unique',
'isin',
'isna',
'isnull',
'item',
'items',
'keys',

'kurt',
'kurtosis',
'last_valid_index',
'le',
'list',
'loc',
'lt',
'map',
'mask',
'max',
'mean',
'median',
'memory_usage',
'min',
'mod',
'mode',
'mul',
'multiply',
'name',
'nbytes',
'ndim',
'ne',
'nlargest',
'notna',
'notnull',
'nsmallest',
'nunique',
'pct_change',
'pipe',
'plot',
'pop',
'pow',
'prod',
'product',
'quantile',
'radd',
'rank',
'rdiv',
'rdivmod',
'reindex',
'reindex_like',
'rename',
'rename_axis',
'reorder_levels',
'repeat',
'replace',
'resample',
'reset_index',
'rfloordiv',
'rmod',
'rmul',
'rolling',
'round',
'rpow',
'rsub',
'rtruediv',
'sample',
'searchsorted',
'sem',

```
'set_axis',
'set_flags',
'shape',
'shift',
'size',
'skew',
'sort_index',
'sort_values',
'squeeze',
'std',
'struct',
'sub',
'subtract',
'sum',
'swaplevel',
'tail',
'take',
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_frame',
'to_hdf',
'to_json',
'to_latex',
'to_list',
'to_markdown',
'to_numpy',
'to_period',
'to_pickle',
'to_sql',
'to_string',
'to_timestamp',
'to_xarray',
'transform',
'transpose',
'truediv',
'truncate',
'tz_convert',
'tz_localize',
'unique',
'unstack',
'update',
'value_counts',
'values',
'var',
'where',
'xs']
```

In [59]:

```
print("="*110)
print("GROUP BY AND SUMMARIZING")
print("="*110)

print("Mean Income by Marital")

income_by_educ = df.groupby('marital')['income'].mean().sort_values(ascending = False)
income_by_educ
```

GROUP BY AND SUMMARIZING

Mean Income by Marital

Out[59]:

marital

MARRIED 43838.108058

DIVORCED 39881.177721

SEPARATED 25903.775730

NEVER MARRIED 25828.277524

WIDOWED 23827.618056

Name: income, dtype: float64

3.5.1. Multiple Grouping by and Summarizing

In [60]:

```
income_by_educ = df.groupby(['sex', 'degree'])['income'].mean()
income_by_educ
```

Out[60]:

sex degree

FEMALE BACHELOR 34269.353010

GRADUATE 52381.502655

HIGH SCHOOL 21630.135693

JUNIOR COLLEGE 22180.077273

LT HIGH SCHOOL 11350.578125

MALE BACHELOR 60334.881241

GRADUATE 82374.545373

HIGH SCHOOL 34749.177272

JUNIOR COLLEGE 43066.722973

LT HIGH SCHOOL 23434.511111

Name: income, dtype: float64

3.5.2. Reset the Index for the Series

In [61]:

```
income_by_educ = df.groupby(['sex', 'degree'])['income'].mean().reset_index()
income_by_educ
```

Out[61]:

	sex	degree	income
0	FEMALE	BACHELOR	34269.353010
1	FEMALE	GRADUATE	52381.502655
2	FEMALE	HIGH SCHOOL	21630.135693
3	FEMALE	JUNIOR COLLEGE	22180.077273
4	FEMALE	LT HIGH SCHOOL	11350.578125
5	MALE	BACHELOR	60334.881241
6	MALE	GRADUATE	82374.545373
7	MALE	HIGH SCHOOL	34749.177272
8	MALE	JUNIOR COLLEGE	43066.722973

	sex	degree	income
9	MALE	LT HIGH SCHOOL	23434.511111

3.5.3. Unstack the Table

In [62]:

```
income_by_educ = df.groupby(['sex', 'marital', 'degree'])['income'].mean().unstack()
income_by_educ
```

Out[62]:

		degree	BACHELOR	GRADUATE	HIGH SCHOOL	JUNIOR COLLEGE	LT HIGH SCHOOL
sex	marital						
FEMALE	DIVORCED		39995.969556	44455.468750	22684.660714	25310.750000	15149.500000
	MARRIED		34316.479846	54501.250000	24494.319813	19109.576087	9131.928571
	NEVER MARRIED		28860.589286	52449.618143	18487.483108	23555.625000	11131.187500
	SEPARATED		88680.000000	40645.000000	18490.395833	24941.250000	8313.750000
	WIDOWED		30021.875000	NaN	18746.691176	NaN	15565.187500
MALE	DIVORCED		56866.050000	109717.882909	39306.581451	61363.392857	30252.812500
	MARRIED		70023.384629	86758.384400	44076.445458	41825.347222	23618.440000
	NEVER MARRIED		39767.437500	43550.613636	21546.168831	34086.375000	19715.464286
	SEPARATED		NaN	68531.425500	15080.218750	20322.500000	16812.250000
	WIDOWED		22862.812500	88680.000000	26683.178571	49882.500000	NaN

3.6. ADVANCED GROUPING AND MULTIPLE AGGREGATION

In [63]:

```
summary = df.groupby('degree').agg({
    'income': ['mean', 'median', 'std', 'var', 'count'],
    'age': ['mean', 'median'],
    'height': ['mean', 'median']
})
summary
```

Out[63]:

	income					age		
	mean	median	std	var	count	mean	median	mean
degree								
BACHELOR	47738.621187	40645.00	38036.482608	1.446774e+09	209	44.550239	44.0	68.057416
GRADUATE	68457.773552	49882.50	47526.992948	2.258815e+09	125	47.624000	47.0	68.176000
HIGH SCHOOL	28309.892098	24017.50	26671.333894	7.113601e+08	491	43.602851	43.0	67.173116

	income					age		
	mean	median	std	var	count	mean	median	mean
degree								
JUNIOR COLLEGE	30580.141304	27712.50	20292.993851	4.118056e+08	92	42.413043	42.5	66.858696
LT HIGH SCHOOL	18412.616883	15703.75	13324.116851	1.775321e+08	77	47.363636	49.0	66.636364

4. FEATURES ENGINEERING

In [64]:

```
df.head()
```

Out[64]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6

4.1. Create Income Categories for the Participants

In [66]:

```
mean_income = df['income'].mean()
print(f"Mean: {mean_income}")
```

Mean: 36887.21835211268

In [68]:

```
df["Income_Category"] = np.where(df["income"] >= mean_income,
                                   "HIGH INCOME", "LOW INCOME")
df.head()
```

Out[68]:

	id	sex	degree	income	marital	age	height	weight	hrswrk	Income_Category
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60	HIGH INCOME
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40	HIGH INCOME
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20	LOW INCOME
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37	LOW INCOME
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6	LOW INCOME

4.2. Frequencies for Income Categories

In [70]:

```
freq = df["Income_Category"].value_counts()
freq
```

```
Out[70]:
Income_Category
LOW INCOME      637
HIGH INCOME     357
Name: count, dtype: int64
```

4.3. Descriptive Statistics

```
In [72]:
df.drop(columns = ['id']).describe()
```

```
Out[72]:
```

	income	age	height	weight	hrswrk
count	994.000000	994.000000	994.000000	994.000000	994.000000
mean	36887.218352	44.488934	67.414487	181.318913	42.640845
std	34702.256068	13.126865	4.037741	41.641615	14.394974
min	369.500000	19.000000	57.000000	90.000000	1.000000
25%	15703.750000	33.000000	64.000000	150.000000	38.000000
50%	27712.500000	44.000000	67.000000	175.000000	40.000000
75%	49882.500000	55.000000	70.000000	205.000000	50.000000
max	158656.952000	79.000000	79.000000	410.000000	89.000000

```
In [74]:
bins = [19, 35, 45, 55, float('inf')]
labels = ["YOUNG", "YOUNG ADULT", "MIDDLE ADULT", "OLDER ADULTS"]
df['Age_Cat'] = pd.cut(df['age'],
                       bins = bins,
                       labels = labels,
                       right = False)
```

```
In [75]:
df.head()
```

```
Out[75]:
```

	id	sex	degree	income	marital	age	height	weight	hrswrk	Income_Category	Age_Cat
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60	HIGH INCOME	MIDDLE ADULT
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40	HIGH INCOME	YOUNG ADULT
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20	LOW INCOME	OLDER ADULTS
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37	LOW INCOME	YOUNG ADULT
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6	LOW INCOME	OLDER ADULTS

In [77]:

```
freq = df['Age_Cat'].value_counts()
freq
```

Out[77]:

```
Age_Cat
YOUNG      273
OLDER ADULTS  257
YOUNG ADULT  234
MIDDLE ADULT  230
Name: count, dtype: int64
```

4.4. FREQUENCY DISTRIBUTION TABLE

In [79]:

```
df.describe(include = ["object", "category"])
```

Out[79]:

	sex	degree	marital	Income_Category	Age_Cat
count	994	994	994	994	994
unique	2	5	5	2	4
top	MALE	HIGH SCHOOL	MARRIED	LOW INCOME	YOUNG
freq	507	491	480	637	273

In [80]:

```
freq = df['degree'].value_counts()
freq
```

Out[80]:

```
degree
HIGH SCHOOL      491
BACHELOR         209
GRADUATE         125
JUNIOR COLLEGE   92
LT HIGH SCHOOL   77
Name: count, dtype: int64
```

In []: